

Hadoop & Spark



Dr. Villanes

Introduction to HDFS and MapReduce



The Motivation for Hadoop

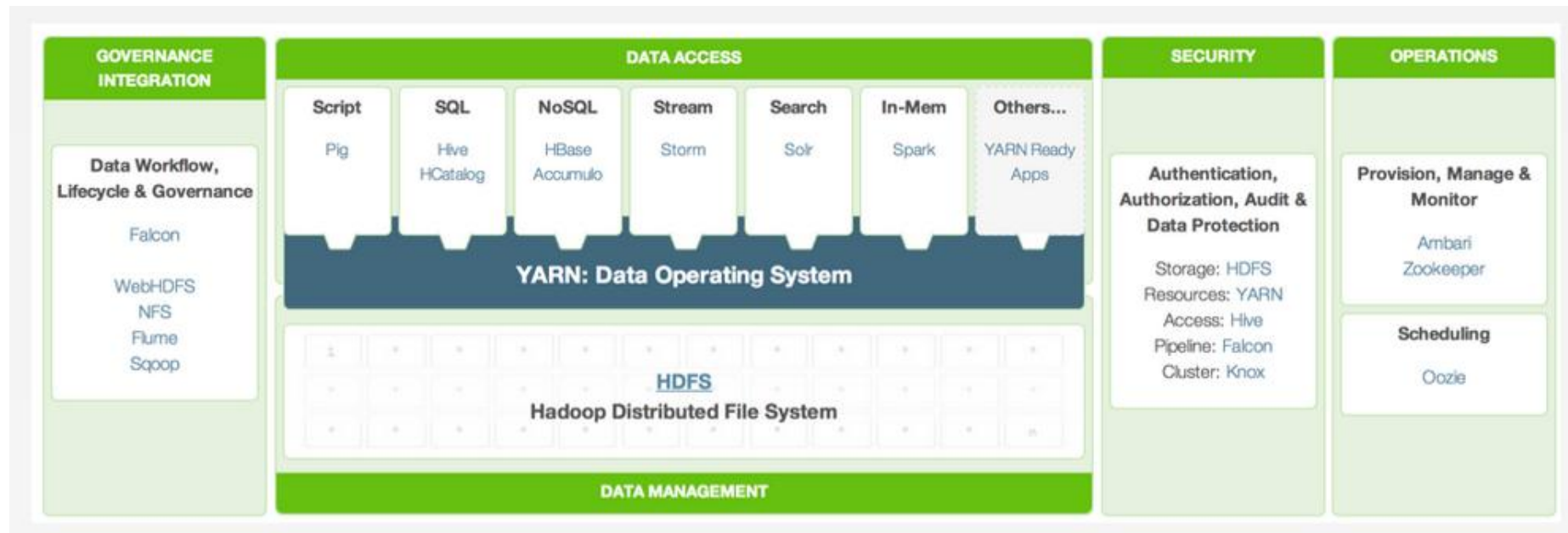
- Velocity
- Variety
- Volume
- Data Has Value
- Two key problems to address:
 - How can we reliably store large amounts of data at a reasonable cost?
 - How can we analyze all the data we have stored?

Core of Apache Hadoop

- 'Core' Hadoop consists of two main components:
 - 1. Storage: the Hadoop Distributed File System (HDFS)**
 - 2. Processing: MapReduce**
- Plus the infrastructure needed to make them work, including:
 - Filesystem and administration utilities
 - Job scheduling and monitoring

What is Apache Hadoop?

- Apache Hadoop has matured and developed to become a **data platform**



YARN: MapReduce 2.0 (MRv2)

Benefit 1: Scalability

Definitions:

- Individual machines are known as nodes
 - Each node both stores and processes data
- A cluster: a set of nodes
- A cluster can have as few as one node to as many as several thousands.
- A set of machines running HDFS and Map Reduce is known as a *Hadoop Cluster*.
- How do we increase performance?: Add more nodes to the cluster to increase scalability
 - You can expect better performance by simply adding more nodes.
 - However, if we keep adding nodes, we are guaranteed that some of them will fail...

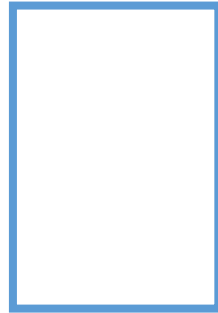
Benefit 2: Fault Tolerance

- Problem: adding nodes increases chances that any one of them will fail
 - **Solution: build redundancy into the system and handle it automatically**
- Files loaded into HDFS are replicated across nodes in the cluster
 - **If a node fails, its data is re-replicated using one of the other copies**
- Data processing jobs are broken into individual tasks
 - Each task takes a small amount of data as input
 - **Thousands of tasks often run in parallel**
 - If a node fails during processing, its tasks are rescheduled elsewhere
- Routine failures are handled automatically without any loss of data

HDFS - Review

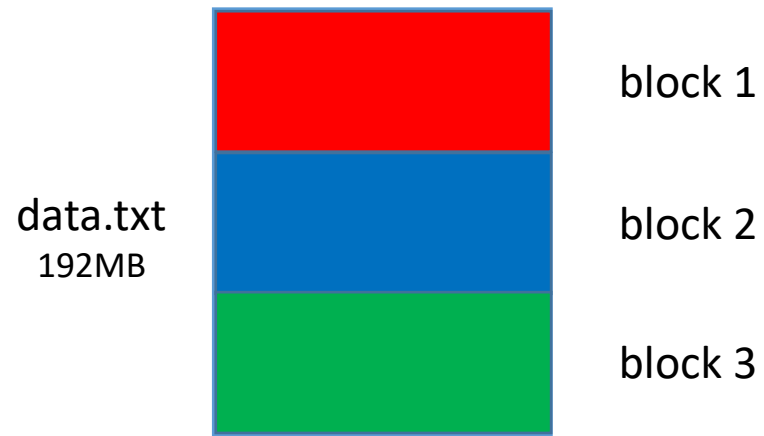
- Files are stored in HDFS. What's going on behind the scenes?
- Imagine we are going to store a file called data.txt in HDFS

data.txt
192MB



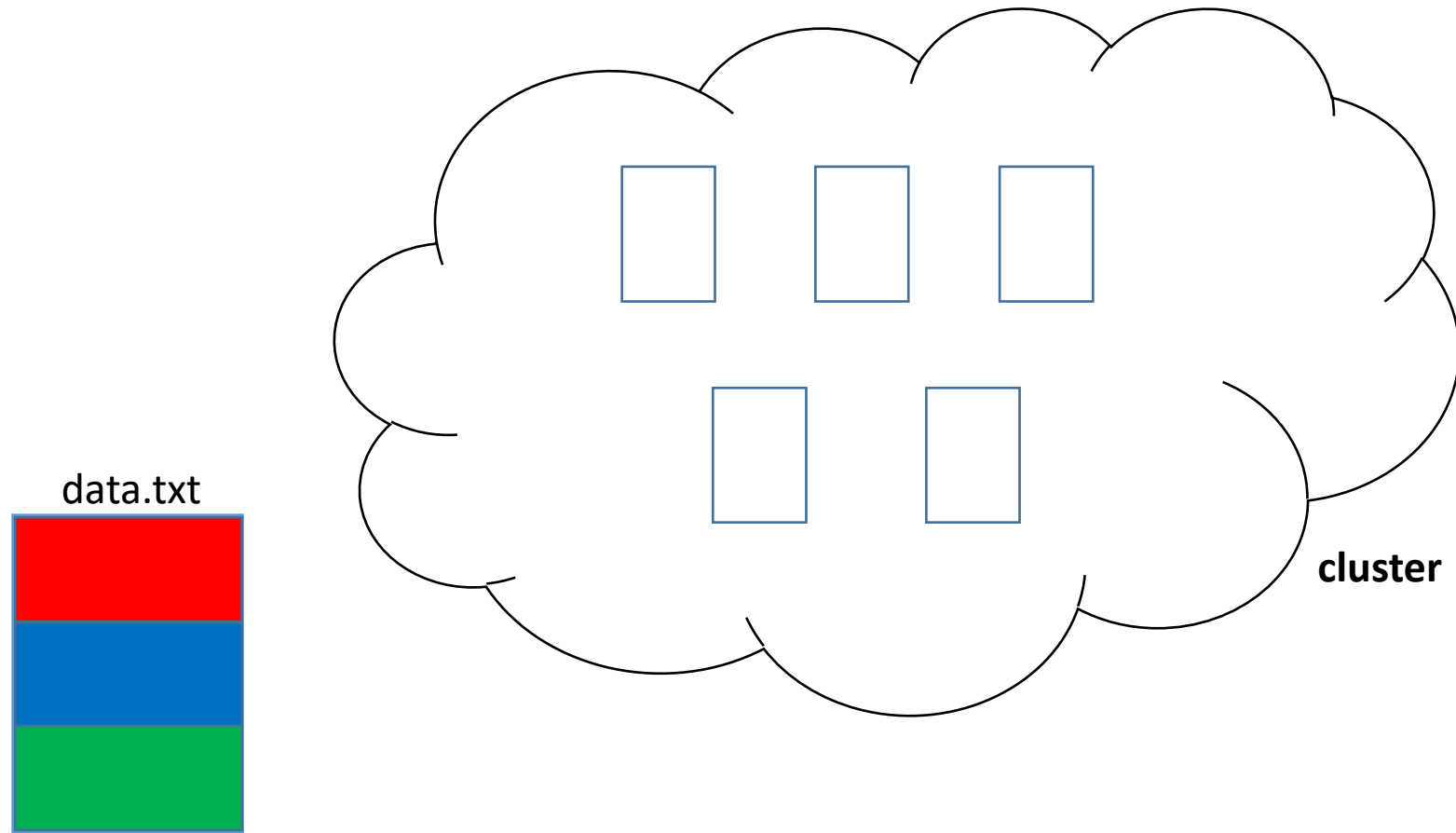
HDFS - Review

- When a file is loaded into HDFS, it is divided into blocks:



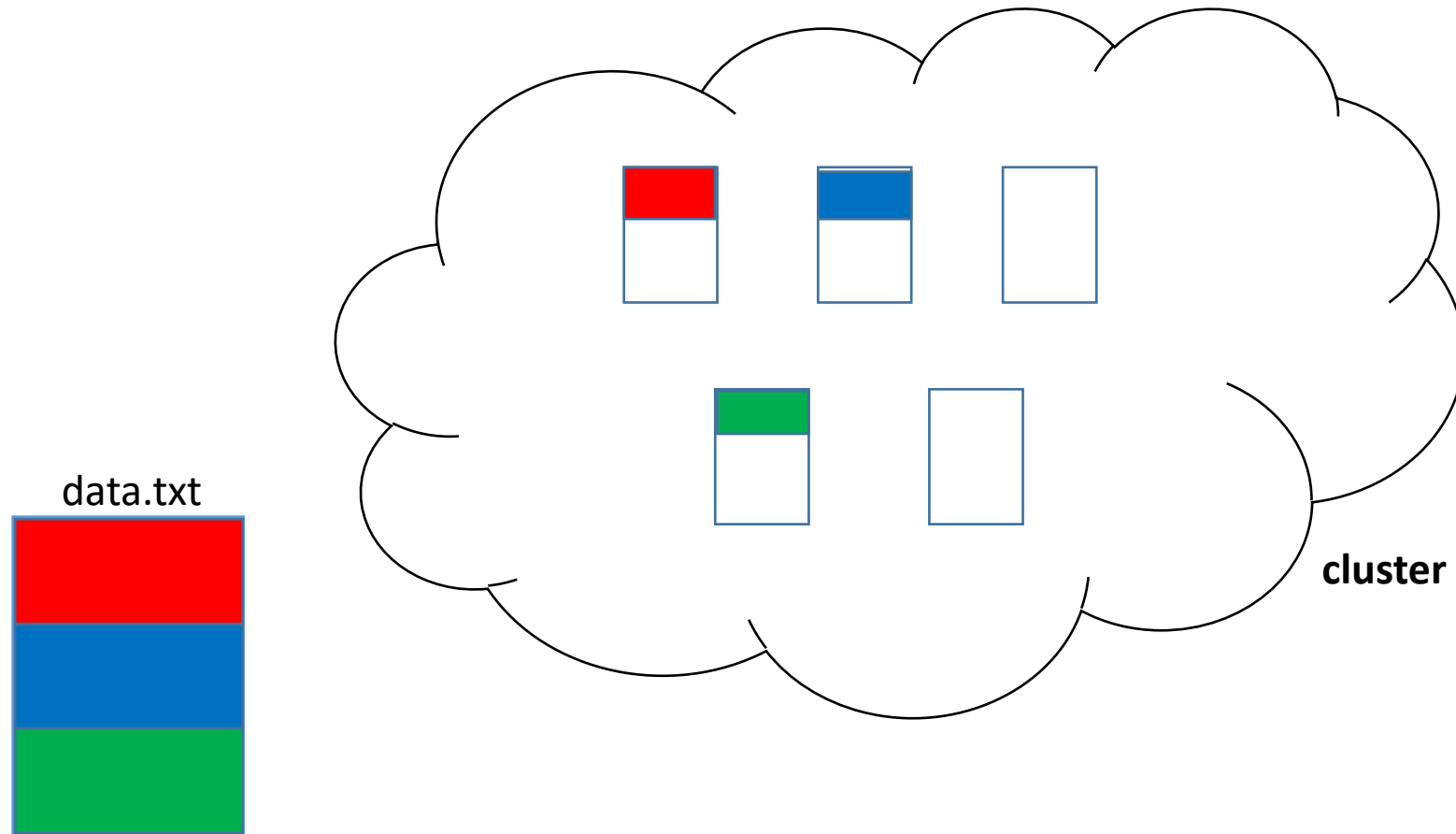
HDFS - Review

- The file is uploaded into HDFS:



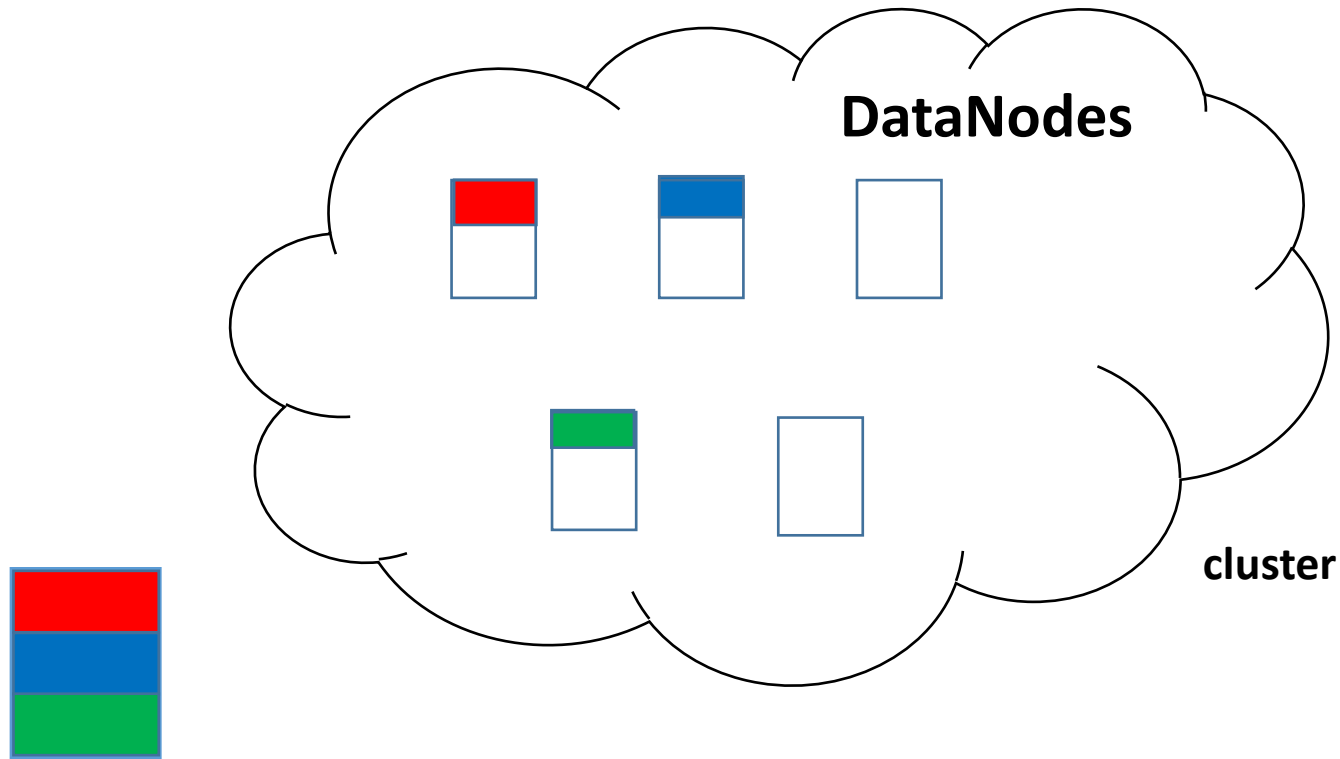
HDFS - Review

- Each block will get stored in one node in the cluster:



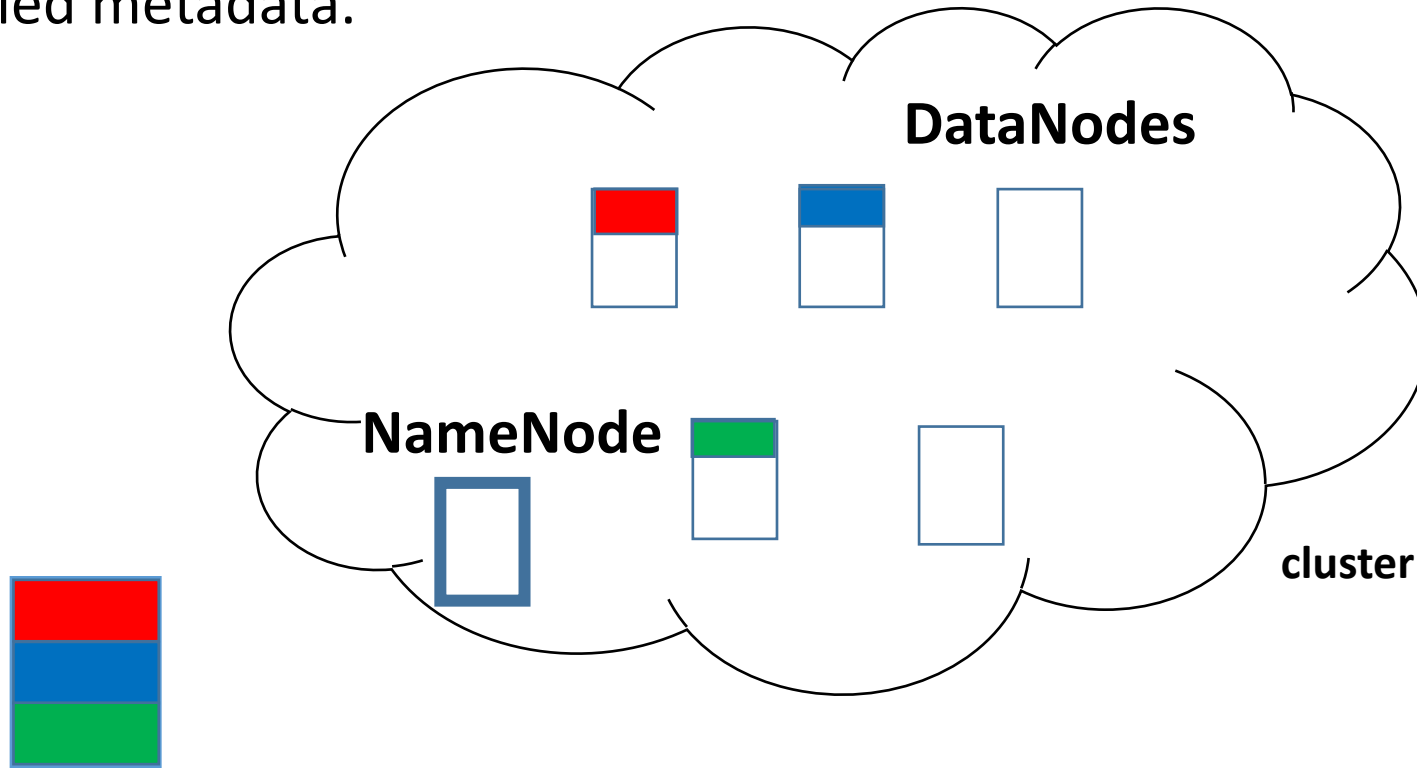
HDFS - Review

- DataNode is responsible for storing the actual data in HDFS:



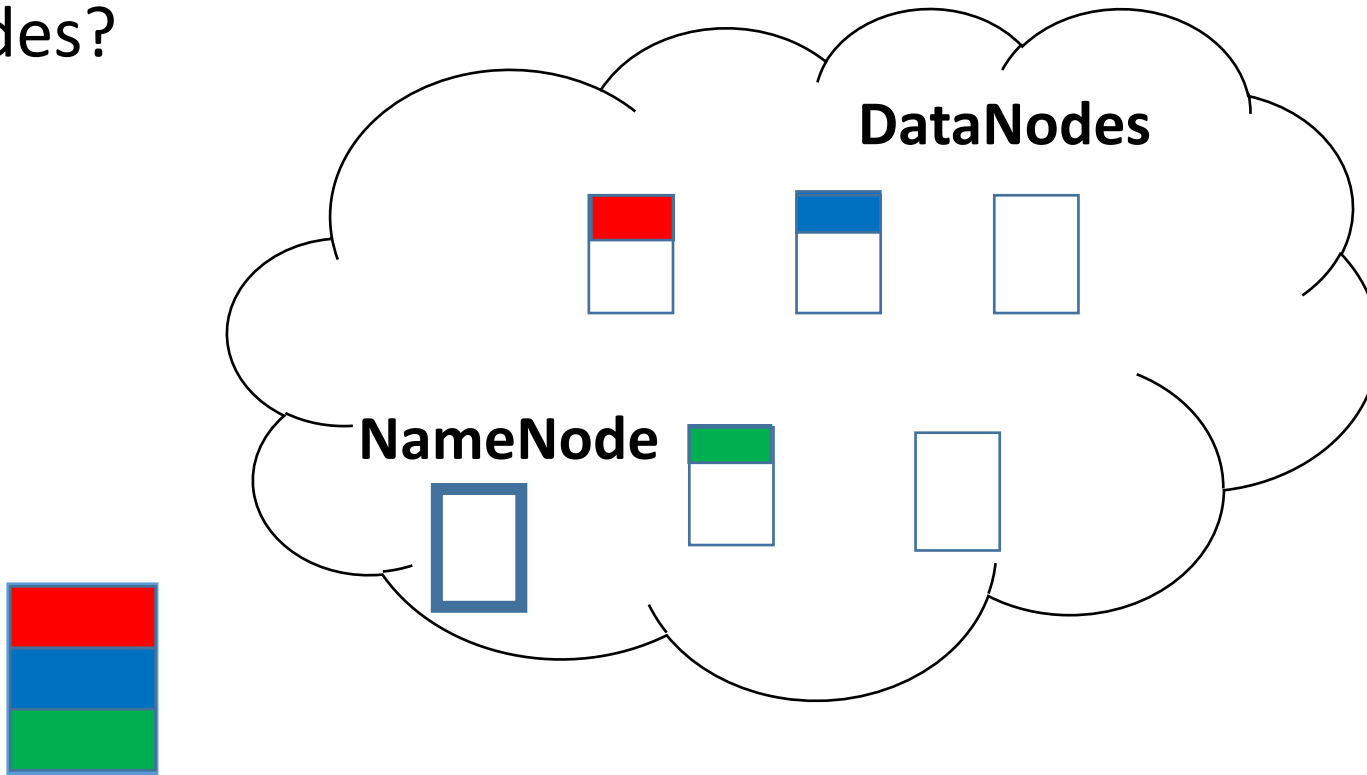
HDFS - Review

- We need to know which blocks make up the original file, and that is handled by a separate node called the NameNode. The information stored in the NameNode is called metadata.



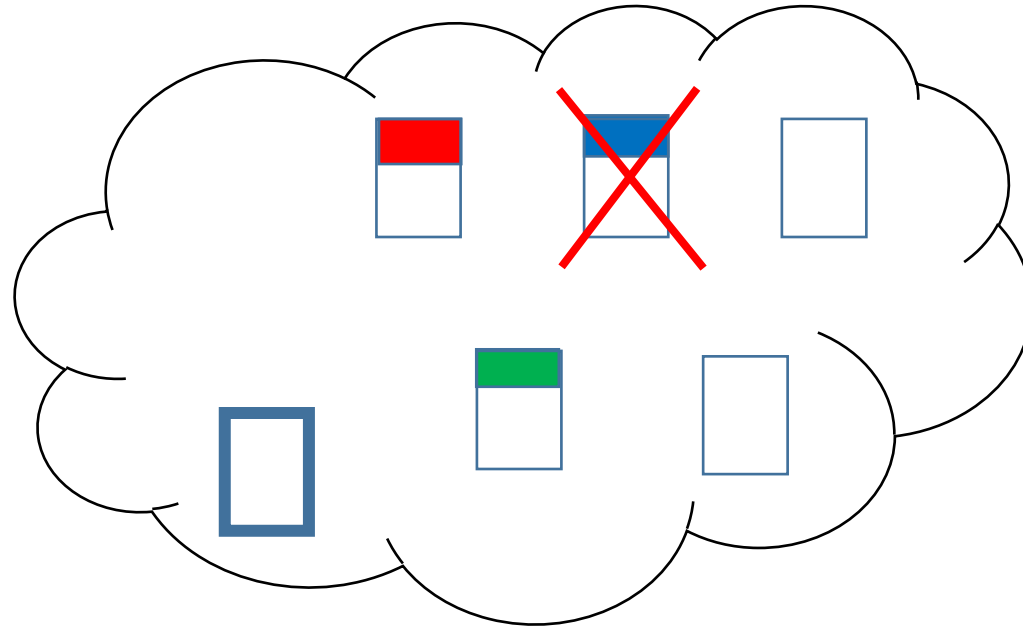
HDFS - Review

- Is there a problem? What happens if we have a disk failure in one of the nodes?



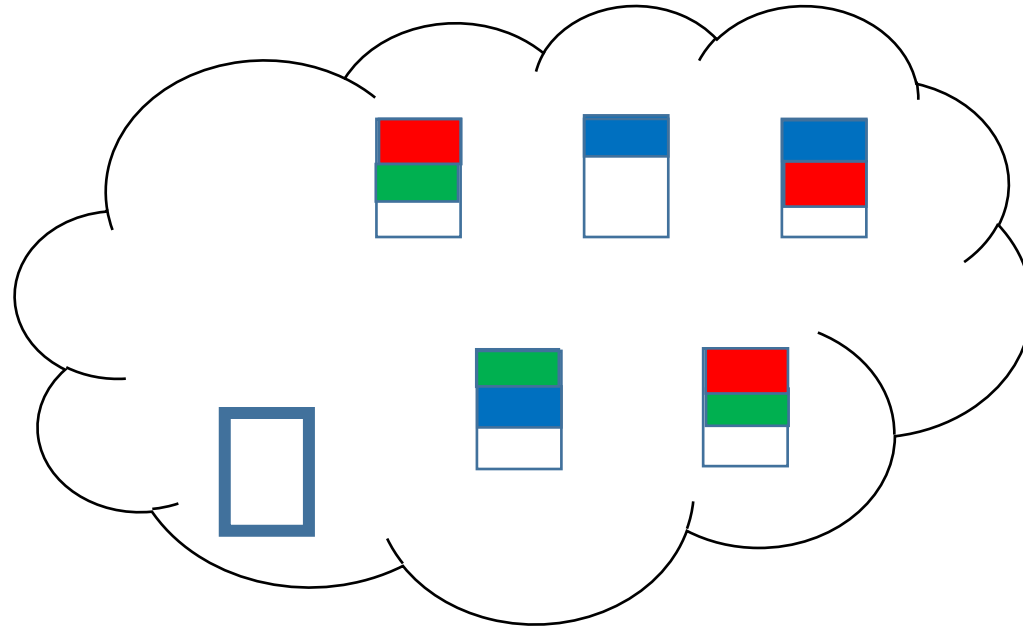
HDFS – Review – Data Redundancy

- The problem with things right now is that if one of our nodes fails, we are left with missing data for the file



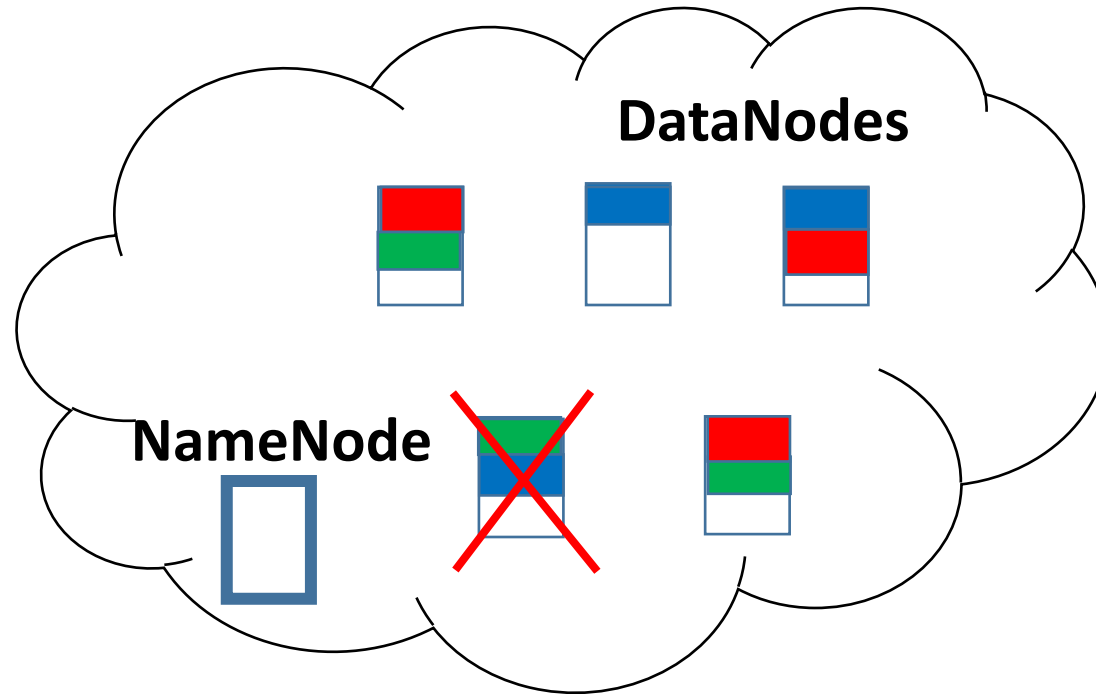
HDFS – Review – Data Redundancy

- To solve this problem, Hadoop replicates each block THREE times as it is stored in HDFS:



HDFS – Review – Data Redundancy

- Now, if a random node fails, it's not a problem because we have two more copies of the block in another node.



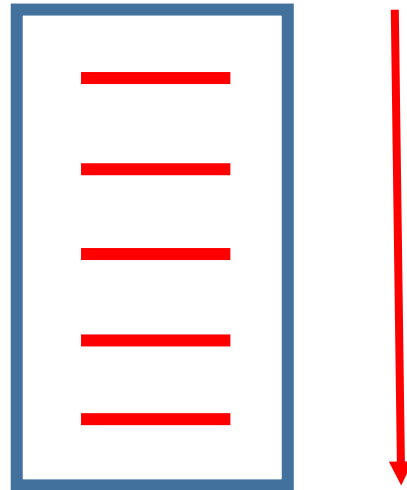
MapReduce

MapReduce

- Method for distributing a task across multiple nodes
- We process data in Hadoop using MapReduce
- Each node processes data stored on that node to the extent possible
- The primary advantages of abstracting your jobs as MapReduce running over a distributed infrastructure are:
 - **Automatic parallelization and distribution of data in blocks across a distributed, scale-out infrastructure**
 - **Fault-tolerance** against failure of storage, compute and network infrastructure
 - **Deployment, monitoring and security capability**

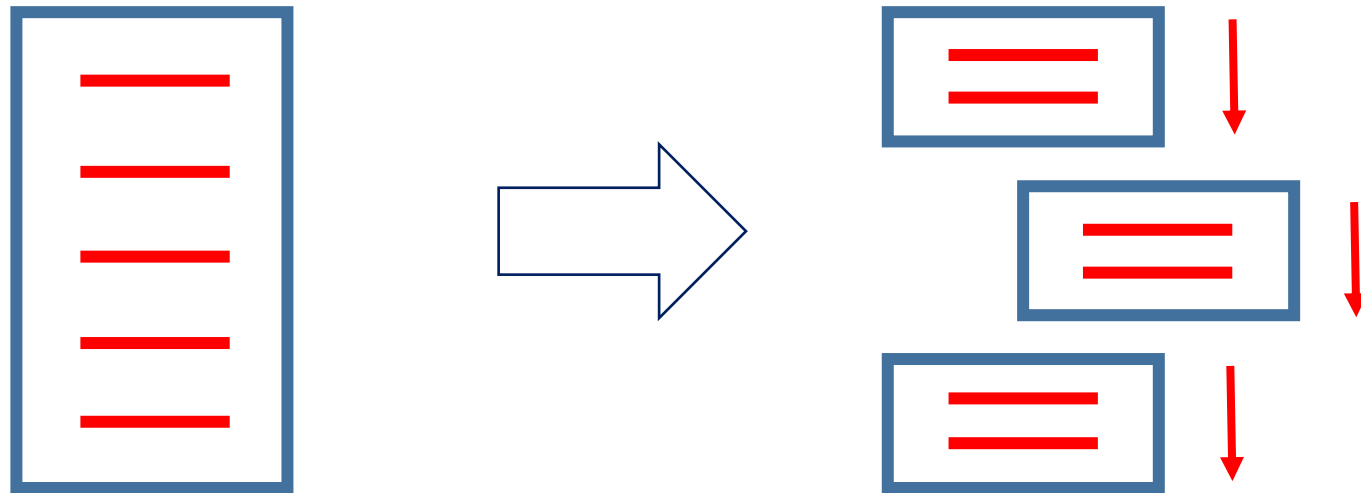
MapReduce example

- Scenario: you have a large file. Processing the file serially, from the top to the bottom, could take a long time.



MapReduce example

- Solution: MapReduce is designed to process data in parallel. Your file is broken into pieces, and then processed in parallels.



MapReduce example

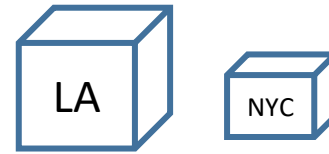
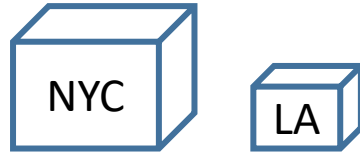
- Objective: given a file, calculate the total sales per city for the year.

Date	City	Category	Amount
01/02/21	NYC	Toys	4.10
01/02/21	Miami	Clothes	21.99
01/01/21	LA	Clothes	25.99
01/01/21	Miami	Music	12.15
01/02/21	NYC	Toys	3.10
01/02/21	Miami	Clothes	50.00
		.	
		.	

MapReduce example



Mappers



NYC



Miami, LA



Reducers

Try that with Java...

```

        reporter.setStatus("OK");
    }

    // Do the cross product and collect the values
    for (String w1 = first) {
        for (String w2 = second) {
            String output = key + ", " + w1 + ", " + w2;
            mr.collect(output, new Text(output));
        }
    }
}

public static class LoadSplits extends MapReduceBase
implements Mapper<Text, Text>, Text, LongWritable {

    public void map(
        Text k,
        Text val,
        OutputCollector<Text, LongWritable> cc,
        Reporter reporter) throws IOException {
        // Find the url
        String line = val.toString();
        int firstColumn = line.indexOf(' ');
        int secondColumn = line.indexOf(':', firstColumn);
        String key = line.substring(firstColumn, secondColumn);
        // drop the rest of the record, I don't need it anyway,
        // just pass it in for the combiner/reducer to sum instead.
        Text outKey = new Text(key);
        mr.collect(outKey, new LongWritable(1L));
    }
}

public static class ReduceUrn extends MapReduceBase
implements Reducer<Text, LongWritable, WritableComparable,
Text> {

    public void reduce(
        Text key,
        Iterator<LongWritable> iter,
        OutputCollector<WritableComparable, Writable> cc,
        Reporter reporter) throws IOException {
        // Add up all the values we see
        long sum = 0;
        while (iter.hasNext()) {
            sum += iter.next().get();
        }
        reporter.setStatus("OK");
        mr.collect(key, new LongWritable(sum));
    }
}

public static class LoadClicks extends MapReduceBase
implements Mapper<WritableComparable, Writable, LongWritable,
Text> {

    public void map(
        WritableComparable key,
        Writable val,
        OutputCollector<LongWritable, Text> cc,
        Reporter reporter) throws IOException {
        mr.collect((LongWritable)val, (Text)key);
    }
}

public static class ClickClicks extends MapReduceBase
implements Reducer<LongWritable, Text, LongWritable, Text> {

    int count = 0;
    public void reduce(
        LongWritable key,
        Iterator<Text> iter,
        OutputCollector<LongWritable, Text> cc,
        Reporter reporter) throws IOException {
        // Only output the first 100 records
        while (count <= 100 && iter.hasNext()) {
            mr.collect(key, iter.next());
            count++;
        }
    }
}

public static void main(String[] args) throws IOException {
    JobConf jc = new JobConf(getClass().getClassLoader(), getClass().getName());
    jc.setJarByClass(TextInputFormat.class);
    jc.setInputFormat(new TextInputFormat());

```

```

    Ip, netOutputPutRegTLabel(Thet, -o, class);
    Ip, netOutputPutValLabel(Thet, -o, class);
    Ip, netRegTLabel(Thet, -o, class);
    Path = "/user/gnome/gnome";
    FileOutputFormat, netOutputPutPath(Ip,
        new Path("/user/gnome/tmp/induced_gnome"));
    Job loadPages = new Job(Ip);
    Ip, netHadoopWriteTask(0);
    Job loadPages = new Job(Ip);

    JobConf jfc = new JobConf(MRExample.class);
    Ip, netJobName("Load and Filter Users");
    Ip, netInputFormat(Thet, InputFormat.class);
    Ip, netOutputPutRegTLabel(Thet, -o, class);
    Ip, netOutputPutValLabel(Thet, -o, class);
    Ip, netMapperClass(LoadAndFilterUsers.class);
    FileOutputFormat, addInputPath(Ip, new
    Path("/user/gnome/user"));
    FileOutputFormat, netOutputPutPath(Ip,
        new Path("/user/gnome/tmp/filtered_users"));
    Ip, netHadoopWriteTask(0);
    Job loadUsers = new Job(Ip);

    JobConf jfc = new JobConf(MRExample.class);
    Ip, netJobName("Join Users and Pages");
    Ip, netInputFormat(Thet, InputFormat.class);
    Ip, netOutputPutRegTLabel(Thet, -o, class);
    Ip, netOutputPutValLabel(Thet, -o, class);
    Ip, netMapperClass(JoinUsersAndPages.class);
    Ip, netHadoopWriteTask(0);
    FileOutputFormat, addInputPath(Ip, new
    Path("/user/gnome/tmp/induced_gnome"));
    FileOutputFormat, addInputPath(Ip, new
    Path("/user/gnome/tmp/filtered_users"));
    FileOutputFormat, addInputPath(Ip, new
    Path("/user/gnome/tmp/induced"));
    Ip, netHadoopWriteTask(0);
    Job joinJob = new Job(Ip);
    JobConf jfc = new JobConf(MRExample.class);
    Ip, netJobName("Group Users");
    Ip, netInputFormat(Thet, InputFormat.class);
    Ip, netOutputPutRegTLabel(Thet, -o, class);
    Ip, netOutputPutValLabel(Thet, -o, class);
    Ip, netOutputPutFormat(Thet, -o, class);
    Ip, netMapperClass(GroupUsers.class);
    Ip, netHadoopWriteTask(0);
    FileOutputFormat, addInputPath(Ip, new
    Path("/user/gnome/tmp/induced"));
    FileOutputFormat, addInputPath(Ip, new
    Path("/user/gnome/tmp/grouped"));
    Ip, netHadoopWriteTask(0);
    Job groupJob = new Job(Ip);
    JobConf jfc = new JobConf(MRExample.class);
    top100, netJobName("Top 100 sites");
    top100, netInputFormat(Thet, InputFormat.class);
    top100, netOutputPutRegTLabel(Thet, -o, class);
    top100, netOutputPutValLabel(Thet, -o, class);
    top100, netOutputPutFormat(Thet, -o, class);
    top100, netMapperClass(CountTop100.class);
    top100, netHadoopWriteTask(0);
    FileOutputFormat, addInputPath(Ip, new
    Path("/user/gnome/tmp/grouped"));
    FileOutputFormat, netOutputPutPath(Ip, new
    Path("/user/gnome/tmp/first100sitesForUser"));
    top100, netHadoopWriteTask(0);
    Job limit = new Job(top100);
    limit, addInputPath(Ip, groupJob);

    JobConf jfc = new JobConf(MRExample.class);
    Ip, netJobName("Find top 100 sites for user");
    Ip, addJob(loadPages);
    Ip, addJob(loadUsers);
    Ip, addJob(joinJob);
    Ip, addJob(groupJob);
    Ip, addJob(limit);
    Ip, run();

```

Pig and Hive: Motivation

- MapReduce code is typically written in Java. Although it can be written in other languages using the Streaming API of Hadoop
- Streaming Requires a programmer who understands how to think in terms of MapReduce, who understands the problem they're trying to solve and who has enough time to write and test the code.
- Meanwhile, **many other people want to analyze data**
- **Needed:** a **higher-level abstraction on top of MapReduce providing the ability to query the data** without needing to know MapReduce intimately.
- **Solution:** Pig and Hive address these needs.

Introduction to Spark



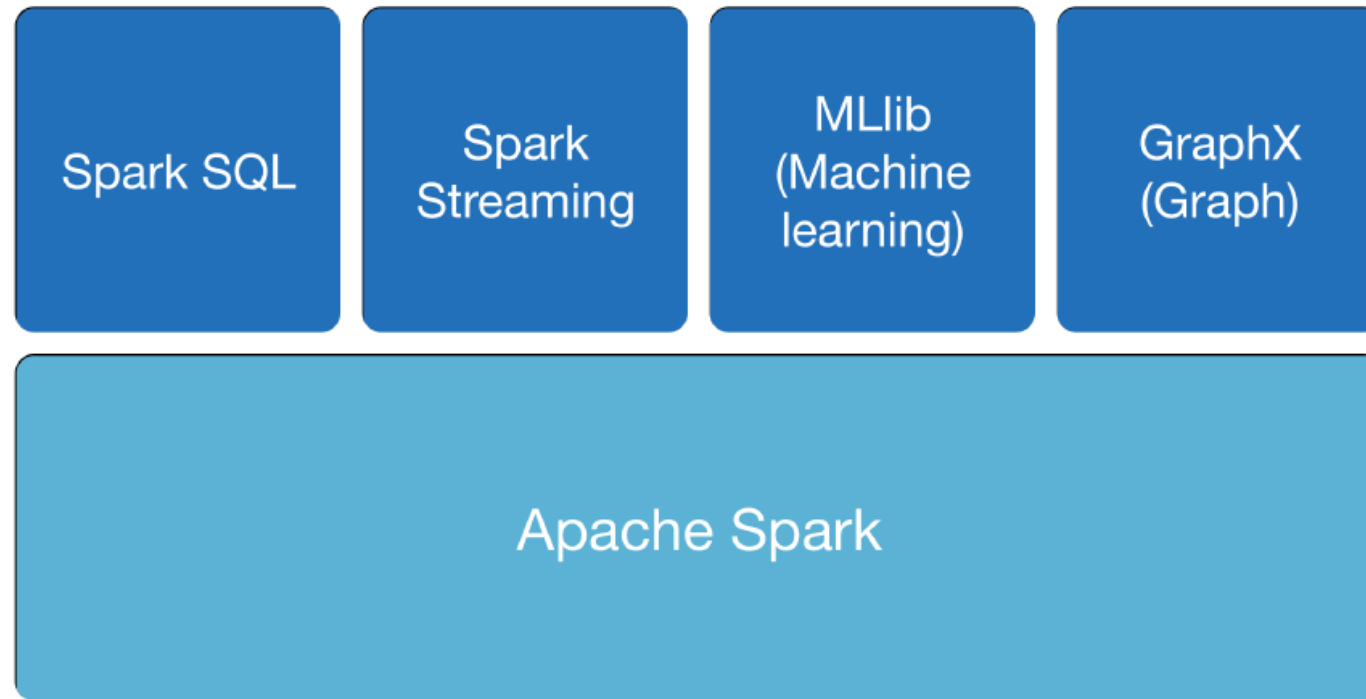
What is Spark?

- Apache Spark is an open-source, cluster computing framework
- Developed with a focus on interactive, iterative computations
 - Utilizes *in-memory processing*
 - Makes it ideal for data science applications (data mining, machine learning)
- Extensive API support for Java, Scala, R and Python
 - You can use it *interactively* from the Scala, Python and R shells



Apache Spark Ecosystem Components

- Combine SQL, streaming, and complex analytics.



Why Use Spark for Data Science?

- **Iteration**

- Spark was designed to facilitate iterative computation

- **Ease of use**

- Multiple available APIs allow for familiar development environments

- **Speed**

- Run programs up to 100x faster than Hadoop MapReduce in memory, or 10x faster on disk.

- **Runs Everywhere**

- Spark runs on Hadoop, standalone, or in the cloud. It can access diverse data sources including HDFS, Cassandra, HBase, and S3.

What is the **relationship** between Spark and Hadoop? (1)

1. Hadoop and Apache Spark are both **big-data frameworks**.
2. They don't serve the same purposes:
 - Hadoop is essentially a distributed data infrastructure: It distributes massive data collections across multiple nodes within a cluster of commodity servers. It also indexes and keeps track of that data, enabling big-data processing and analytics far more effectively than was possible previously.
 - Spark is a data-processing tool that operates on those distributed data collections; it doesn't do distributed storage.
3. Spark *does not* provide storage
 - Spark can create distributed datasets from any storage source supported by Hadoop, including your local file system, HDFS, etc.
4. You can use one without the other
 - Hadoop includes storage component (HDFS), but also the processing component (MapReduce): so you don't need Spark to get your processing done.
 - Conversely, you can also use Spark without Hadoop. **Spark does not come with its own file management system**, though, so it needs to be integrated with one -- if not HDFS, then another cloud-based data platform.

What is the **relationship** between Spark and Hadoop? (2)

5. Spark is speedier:

- MapReduce operates in steps, Spark operates on the whole data set in one fell swoop.
- MapReduce workflow: read data from the cluster, perform an operation, write results to the cluster, read updated data from the cluster, perform next operation, write next results to the cluster, etc.
- Spark: read data from the cluster, perform all of the requisite analytic operations, write results to the cluster, done.

6. MapReduce and Spark both provide a framework for scalable data processing

- Both platforms provide fault tolerance

What is the **relationship** between Spark and Hadoop? (3)

7. MapReduce uses only two fundamental data processing operations, while Spark supports over 80 distinct operations
 - Most machine-learning algorithms, for example, require multiple operations
8. Disk read/write vs. in-memory processing
 - MapReduce must write to and read from disk between operations
 - Spark utilizes in-memory processing to avoid intermediate reads/writes

PySpark

Before we go into PySpark....



Apache Spark is an open-source cluster-computing framework, built around speed, ease of use, and streaming analytics

Ease of Use

Write applications quickly in Java, Scala, Python, R, and SQL.

Python + Spark = PySpark

- PySpark is the collaboration of Apache Spark and Python.
- **Apache Spark** is an open-source cluster-computing framework, built around speed, ease of use, and streaming analytics.
- **Python** is a general-purpose, high-level programming language.



```
df = spark.read.json("logs.json")
df.where("age > 21")
  .select("name.first").show()
```

Spark's Python DataFrame API

PySpark Core Concepts

Two important core concepts:

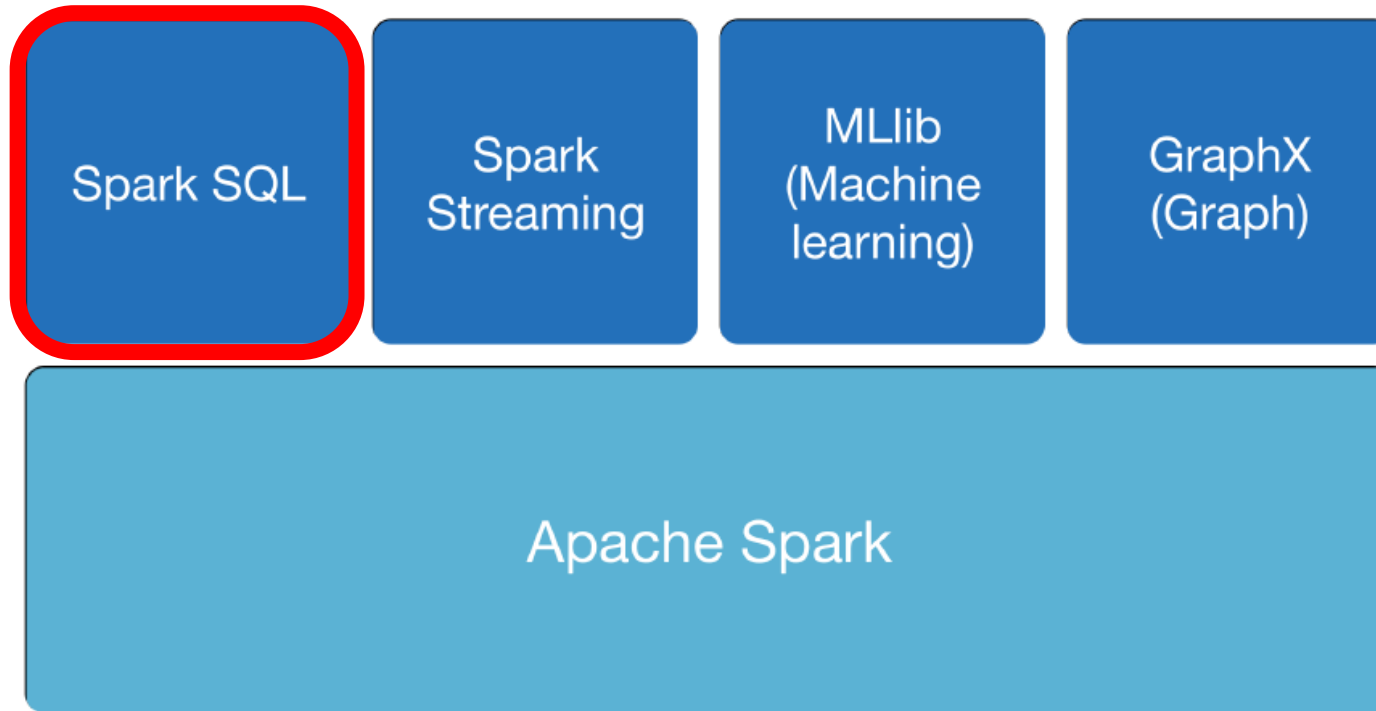
- **SparkSession**: Main entry point for DataFrame and SQL functionality.
- **Dataframe**: A distributed collection of data grouped into named columns.

Documentation: <https://spark.apache.org/docs/latest/api/python/index.html>

Spark SQL

Apache Spark Components

- Combine SQL, streaming, and complex analytics.



Apache Spark SQL



- Spark SQL is a Spark's module for working with structured data.
- There are several ways to interact with Spark SQL including SQL, the DataFrames API and the Datasets API. However, PySpark only implements the DataFrames API.
- A DataFrame is a distributed collection of data organized into named columns. It is **conceptually equivalent to a table in a relational database or a data frame in R/Python**, but with richer optimizations under the hood. DataFrames can be constructed from a wide array of [sources](#) such as: structured data files, tables in Hive, external databases, or existing RDDs.
- Executes SQL queries written using either a basic SQL syntax or HiveQL.

PySpark & Spark SQL Cheat Sheet

Spark SQL is Apache Spark's module for working with structured data.



Initializing SparkSession

A **SparkSession** can be used to create **DataFrame**, register **DataFrame** as views, execute SQL over views, and read csv, json, txt and parquet files.

In EMR, **SparkSession** is automatically created for you. The **SparkSession** is accessible through a variable called *spark*. If you want to verify your Spark version:

```
>>> spark.version
```

Creating DataFrames

From Spark Data Sources

CSV

```
>>> df = spark.read.csv("s3a://bucket_name/airlines.csv", inferSchema = True, header=True)
```

JSON

```
>>> df2 = spark.read.json("s3a://bucket_name/customer.json")
>>> df2.show()
```

```
>>> df3 = spark.read.load("s3a://bucket_name/people.json", format="json")
```

Parquet files

```
>>> df4 = spark.read.load("s3a://bucket_name/users.parquet")
```

TXT files

```
>>> df5 = spark.read.text("s3a://bucket_name/people.txt")
```

View the DataFrame

Show() - Displays the top 20 rows of **DataFrame** in a tabular form.

```
>>> df.show()
```

Show(n) - Displays the top *n* rows of **DataFrame** in a tabular form.

```
>>> df.show(n)
```

Inspect Data

```
>>> df.describe().show()
>>> df.columns
>>> df.count()
>>> df.distinct().count()
>>> df.printSchema()
```

Compute summary statistics
Return the columns of df
Count the number of rows in df
Count the number of distinct rows in df
Print the schema of df

Queries

```
>>> from pyspark.sql import functions as F
Select
>>> df.select("firstName").show()
>>> df.select("firstName", "lastName") \
    .show()
>>> df.select(df["firstName"], df["age"] + 1)
>>> df.select(df["age"] > 24).show()
```

Show all entries in *firstName* column
Show all entries in *firstName*, and *lastName*
Show all entries in *firstName* and *age*, add 1 to the entries of *age*
Show all entries where *age* > 24

When

```
>>> df.select("firstName",
    F.when(df.age > 30, 1) \
    .otherwise(0)) \
    .show()
>>> df[df.firstName.isin ("Jane", "Boris")]
    .show()
```

Show *firstName* and 0 or 1 depending on *age* > 30

Show *firstName* if in the given options

Like

```
>>> df.select("firstName",
    df.lastName.like ("Smith")) \
    .show()
```

Show *firstName*, and *lastName* if *lastName* is like Smith

Startswith – Endswith

```
>>> df.select("firstName",
    df.lastName \
    .startswith("Sm")) \
    .show()
>>> df.select(df.lastName.endswith("th")) \
    .show()
```

Show *firstName*, and *lastName* if *lastName* starts with *Sm*

Show last names ending in *th*

Substring

```
>>> df.select(df.firstName.substr(1, 3) \
    .alias("name")) \
    .collect()
```

Return substrings of *firstName*

Between

```
>>> df.select(df.age.between(22, 24)) \
    .show()
```

Show *age* if values between 22 and 24

Adding Columns

```
>>> from pyspark.sql.functions import log
```

```
>>> df2 = df.withColumn("new_column", log("rating"))
>>> df2.show()
```

Duplicate Values

```
>>> df = df.dropDuplicates()
```

GroupBy

```
>>> df.groupBy("age") \
    .count() \
    .show()
```

Group by *age*, count the members in the groups

Filter

```
>>> df.filter(df["age"] > 24).show()
```

Filter entries of *age*, only keep those records of which the values are > 24

Missing & Replacing Values

```
>>> df.na.fill(50).show()
>>> df.na.drop().show()
>>> df.na \
    .replace(10, 20) \
    .show()
```

Replace null values
Return new df omitting rows with null values
Return new df replacing one value with another

Running SQL Queries Programmatically

Registering DataFrames as Views

```
>>> df.createOrReplaceTempView("customer")
```

Query Views

```
>>> sqlDF = spark.sql("SELECT * FROM customer").show()
```

Output

Data Structure

```
>>> rdd1 = df.rdd
>>> df.toPandas()
```

Convert *df* into an RDD
Return the contents of *df* as Pandas *DataFrame*

Write & Save to Files

```
>>> df.select("firstName", "city") .write.save("nameAndCity.parquet")
>>> df.select("firstName", "age") .write .save("namesAndAges.json", format="json")
```